

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №4

Выполнил:

Алексеев Андрей

Группа: К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

В рамках лабораторной работы требовалось выполнить развёртывание backend-приложения на удалённом сервере. Для этого необходимо было предварительно подготовить серверное окружение, установить зависимости, перенести проект на сервер, запустить приложение и настроить nginx в режиме обратного прокси для доступа к приложению извне.

В качестве разворачиваемого приложения использовался микросервисный стек lab23. Внешней точкой входа в приложение был выбран api-gateway, который работает на порту 8080. Доступ извне должен осуществляться не напрямую к контейнеру, а через nginx, который принимает HTTP-запросы на сервере и перенаправляет их во внутренний gateway.

Исходные данные

Для выполнения работы использовался удалённый сервер с адресом 158.160.152.229. Подключение к серверу выполнялось по SSH под пользователем timur1516 с использованием ключа из каталога lab4. Исходный проект находился в локальном репозитории и передавался на сервер в каталог ~/apps/itmo-ac-s-backend.

Стек lab23 включает общий docker-compose.yml, набор Dockerfile для сервисов, api-gateway, девять доменных микросервисов, Kafka, Elasticsearch и PostgreSQL-контейнеры. Такой состав позволил развернуть не отдельное приложение, а полный backend-контур с инфраструктурными зависимостями.

Для автоматизации и воспроизводимости деплоя также были подготовлены инфраструктурные файлы: конфигурация nginx infra/nginx/lab23.conf, скрипт настройки nginx scripts/lab23/configure_nginx.sh и скрипт удалённого деплоя scripts/lab23/deploy_remote.sh.

Ход работы

На первом этапе был выполнен первичный осмотр репозитория. Были найдены материалы lab4, ключи для подключения к серверу и каталог lab23, содержащий микросервисный compose-стек. После анализа структуры проекта была определена основная точка входа в приложение: сервис api-gateway на порту 8080.

Далее были настроены права доступа к SSH-ключам. Для приватного ключа были выставлены права 600, для публичного ключа — 644. После этого было выполнено тестовое подключение к серверу 158.160.152.229, проверены имя хоста, текущий пользователь, параметры системы и наличие необходимых программ.

После проверки подключения была проведена предварительная настройка удалённого сервера. На сервере были установлены git, docker, docker compose plugin, nginx и curl. Сервисы docker и nginx были включены через systemctl, чтобы они автоматически запускались и были доступны для дальнейшего развёртывания приложения.

Следующим шагом исходный код проекта был передан на сервер. Для этого локальный репозиторий был упакован в архив без каталога .git и распакован на сервере в директорию ~/apps/itmo-acс-backend. Такой способ позволил перенести не только исходный код сервисов, но и инфраструктурные файлы, скрипты и compose-конфигурацию.

После передачи файлов были выставлены права на выполнение для скриптов scripts/lab23/*.sh. Затем был запущен скрипт configure_nginx.sh, который применил подготовленную конфигурацию nginx для приложения lab23. В результате nginx стал работать как reverse проху: внешний HTTP-

трафик поступает на сервер, после чего перенаправляется на локальный api-gateway по адресу 127.0.0.1:8080.

Такой вариант настройки нужен для того, чтобы наружу была открыта только единая точка входа. Внутренние микросервисы, базы данных, Kafka и Elasticsearch не публикуются напрямую для внешних клиентов. Запросы сначала проходят через nginx, затем попадают в api-gateway, а дальше уже распределяются внутри микросервисного контура.

После настройки nginx был выполнен запуск приложения через docker compose. Команда `docker compose up -d --build --remove-orphans` запустила сборку и поднятие контейнеров в фоновом режиме. В процессе первого запуска была выявлена проблема сетевого таймаута Gradle wrapper внутри Docker build. Для устранения этой проблемы Dockerfile были скорректированы через `GRADLE_OPTS` с увеличенными HTTP timeout, после чего сборка была запущена повторно.

Дополнительно был подготовлен контур автоматического деплоя через GitHub Actions. Для этого в репозиторий были добавлены секреты `LAB23_SSH_HOST`, `LAB23_SSH_USER` и `LAB23_SSH_KEY`. При анализе первого падения `deploy job` была найдена причина: стандартного `command_timeout` в 10 минут оказалось недостаточно для удалённой пересборки всех сервисов. После этого таймаут workflow был увеличен до 40 минут.

Проверка результата

После завершения сборки была выполнена проверка фактического состояния контейнеров. Все контейнеры lab23 были подняты, nginx находился в активном состоянии, а внешний адрес `http://158.160.152.229`

отвечал через настроенный reverse проху. Это подтвердило, что приложение доступно извне не напрямую, а через nginx.

Для серверного тестирования был подготовлен отдельный Postman environment `lab23-server.postman_environment.json`, в котором `baseUrl` был установлен как `http://158.160.152.229/api/v1`. Это позволило прогнать ту же коллекцию тестов не против локального окружения, а против приложения, развёрнутого на удалённом сервере.

Финальная end-to-end проверка выполнялась через `newman`. Коллекция `lab23-microservices.postman_collection.json` была успешно запущена в контейнере `postman/newman:alpine`. По итогам проверки было выполнено 65 requests и 65 assertions, при этом количество failed равно 0. Следовательно, API после деплоя на сервер работало корректно.

Итоговое состояние после выполнения работы: SSH-доступ настроен, `docker` и `docker compose` установлены, `nginx` установлен и включён, микросервисный стек `lab23` поднят, reverse проху для `api-gateway` работает, внешний доступ к приложению подтверждён, серверный прогон тестов завершился без ошибок.

Вывод

В результате лабораторной работы было выполнено развёртывание микросервисного backend-приложения `lab23` на удалённом сервере. Сервер был предварительно подготовлен: установлены необходимые зависимости, настроены `docker`, `docker compose` и `nginx`, а также проверено подключение по SSH.

Приложение было передано на сервер, собрано и запущено через `docker compose`. Для внешнего доступа был настроен `nginx` в режиме обратного прокси, который перенаправляет входящие HTTP-запросы на `api-gateway`.

Такой подход позволяет оставить внутренние сервисы закрытыми от прямого внешнего доступа и использовать gateway как единую точку входа в систему.

Работоспособность развёрнутого приложения была подтверждена проверкой контейнеров, запросом к health endpoint и прогоном Postman-коллекции через newman. Все 65 запросов и 65 проверок завершились успешно, без ошибок. Следовательно, цель лабораторной работы была достигнута: приложение развёрнуто на удалённом сервере и доступно извне через nginx reverse proxy.